1. Incremental loading into BigQuery (e.g. MERGE/upsert), partitioning and clustering, and how you would keep query cost down as the data grows to tens of millions of rows.

- Each daily raw file lands in a Cloud Storage bucket. Then a BigQuery load job appends it to a raw table, partitioned by load date. A staging query cleans that day's rows and keeps the newest version per booking. A MERGE then puts them into the fact table: The fact table is partitioned by check-in month and clustered by property, channel and status, which match the common queries.
- To keep query costs down when the data, I would recommend selecting only needed columns and limit the MERGE to the months in the new batch, so it doesn't rescan the whole table;

2. Orchestration, scheduling, and how you would monitor pipeline health and catch data-quality regressions before they reach reporting.

- Airflow will run a single data flow a day that loads raw data , does data-quality checks, MERGE, checks on the results and then refresh the reporting tables. It starts when the file arrives, with a daily schedule as a backup.
- To catch bad data, the SQL validation checks run, so if one fails then the flow stops before the reporting tables are refreshed. This way so a bad batch of data would never reach the dashboard.
- To monitor, each run would save its merge audit and data quality results to a BigQuery table. Alerts will be triggered when a task fails or no file arrives by a scheduled time.

3. Version control, testing and CI/CD, plus documentation/SOPs and lineage for long-term maintainability.

- All code in Git, with pull requests need to be reviewed and approved before merging to the main branch for version control. This also allows you revert back to the previous working version if the main version breaks.
- dbt for the SQL models gives documentation and lineage. I think it’s a great tool for documentation/SOPs and lineage because it builds a visual graph showing how data flows into tables and generates documentation directly from your schema.

4. Key assumptions and trade-offs given the time limit. A note on how you would approach RAG/AI enrichment of this data is welcome but optional.

- Assumptions
    - FX uses the rate on the created_at date
    - Cancelled bookings earn nothing
    - Revenue is reported by check-in month
- Trade-Offs
    - Bookings created after their file's date are only warned about.
    - Rows are validated once before the merge.
- AI Enrichment.
    - An LLM could suggest mappings for new currencies. A person approves each one before it goes into the config, so a model never changes revenue figures directly.